Project Report: NYC Mobility Data Pipeline

Project Team:
Beatrice Bigoni (Mat. 879938), Clelia Meloni (Mat. 895982), Luca Rossini (Mat. 890887)

Abstract

The project developed a Python Data Managment pipeline for analyzing New York City urban mobility (Citi Bike), integrating three heterogeneous data sources: local transactional files, historical weather data via REST API (Open-Weather), and NBA sports calendar acquired via dynamic Web Scraping (Selenium). The implemented workflow manages the entire data lifecycle: from robust capture (with caching and retry logic), to temporal cleansing and alignment (time zone management and granular rounding), to enrichment by expanding the event time window. The final result is a historicized Analytical Base Table on SQLite DBMS, which allowed us to quantify the correlations between exogenous variables and demand. Code analysis highlights how temperature is the main positive driver (corr. 0.41) and how home sporting events (Knicks) generate an average increase in demand compared to the baseline scenario.

1 Introduction

Urban mobility in New York City represents one of the most complex and dynamic logistics systems in the world, where transportation efficiency depends on a delicate balance between infrastructure, environmental conditions, and social events. In recent years, bike sharing — specifically the *Citi Bike* system — has established itself not only as a sustainable alternative, but also as a critical component of the city's connective tissue. However, demand for this service is far from static; it fluctuates in response to external variables that extend well beyond simple daily commuting patterns.
The role of structural factors (such as the topological dislocation of stations) is an acquired fact. Nevertheless, the central hypothesis shifts attention to the incidence of dynamic exogenous variables. In particular, the model is based on the assumption that adverse weather conditions act as a disincentive to Bike Sharing usage. In parallel, it is hypothesized that major events - such as the New York Knicks home games at Madison Square Garden – stimulate demand, resulting in traffic volumes that exceed typical levels.

This scenario leads to the formulation of our central research questions, modelled on the demand-driven approach typical of modern Data Lakes.

2 Objectives e Research Question

To address the complexity of urban mobility and validate the hypotheses formulated, this project adopts a rigorous methodology based on the Data Lifecycle. The primary

technical goal is to engineer an automated pipeline capable of harmonizing structurally heterogeneous sources: bike-sharing transactional logs, meteorological time series, and semi-structured data related to sporting events. By implementing Schema Integration and data cleansing logics, the work aims to transform the raw data into a integrated dataset. Specifically, the project aims to validate whether the integration of exogenous variables (Knicks NBA Weather and Events) provides a statistically relevant "signal" for predicting the demand classes (Low, Medium, High) of the CitiBike service. This approach allows us to overcome the fragmentation of the original information silos to promptly answer the following research questions:

- **Primary research question**: *To what extent do meteorological variables (e.g., precipitation, temperature...) influence the hourly volume of trips and the distribution of demand classes?*
- **Secondary research question**: *Does the presence of major sporting events (Knicks games) generate a measurable alteration in demand compared to usual patterns of service use?*

3 Data acquisition phase

Data acquisition is the process of acquiring and importing data for analysis, taking data from various sources such as databases, files, or web APIs. In accordance with project requirements, three heterogeneous data sources are integrated using different tools based on the nature of the source.

CitiBike transactional data is made available as compressed monthly archives (*.zip*) containing *.csv* files. In the project contest, it is assumed that such files have already been downloaded locally to a working directory, consistent with a batch-oriented approach.

From a data lifecycle perspective, this source has a strong *volume* component: each file can contain millions of records, and over a multi-month time horizon, monolithic memory loading is impractical on local resources. In early development iterations, full import attempts caused the kernel to crash due to RAM saturation, requiring explicit separation between the acquisition phase and the cleanup/transformation phase.

CitiBike data acquisition is therefore implemented as an automated file detection and selection procedure, divided into the following phases:

1. **Identifying** the CSV files available in the working directory via pattern matching (`*-citibike-tripdata_*.csv`),
2. **Exclusion** explanation of files not traceable to the CitiBike source (e.g. analysis output or files related to other sources),
3. **Preparation** of the list of files to be iteratively processed in subsequent stages of the pipeline.

This strategy avoids contamination between raw and derived data and consensus safe pipeline re-execution during debugging and refinement phases.
The raw dataset presented the following features:

- *ride_id*: Unique identifier for the trip (Primary Key) (String).

- *rideable_type*: Type of bike used (e.g., *classic_bike*, *electric_bike*) (Categorical).
- *started_at*: Start timestamp of the trip (unlock time) (Datetime).
- *ended_at*: End timestamp of the trip (lock time) (Datetime).
- *start_station_name*: Name of the station where the trip started (String).
- *start_station_id*: Unique identifier of the start station (String).
- *end_station_name*: Name of the station where the trip ended (String).
- *end_station_id*: Unique identifier of the end station (String).
- *start_lat* / *_lng*: GPS coordinates (WGS84) of the start location (Float).
- *end_lat* / *_lng*: GPS coordinates of the end location (Float).
- *member_casual*: User type: *Member* (annual subscriber) or *Casual* (single ride/day pass) (Categorical).

Not all variables in the raw dataset are required for temporal aggregate demand analysis. Granular spatial information (station ID, Lat, Lng coordinates, and station names) and trip identifier (*ride_id*) were excluded at load time to optimize RAM usage, keeping only time dimensions (*started_at*, *ended_at*) and user segmentation (*member_casual*).

3.1 Weather Data Acquisition (REST API)

From a theoretical point of view, an API (*Application Programming Interface*) represents a standardized interface that allows heterogeneous software systems to communicate with each other. In the context of the project, the **Open-Meteo** REST API was used to enrich the dataset with historical environmental variables (i.e., temperature, precipitation, wind).

Unlike static CSV files, using an API introduces challenges related to network latency and request limits (*Rate Limiting*). To mitigate these risks and ensure reproducibility, a robust acquisition strategy based on two layers has been implemented::

- **Persistent Caching:** Using the *requests_cache* library, API responses are saved locally to a SQLite database (*.cache*). The *expire_after = -1* parameter ensures that historical data, once downloaded, never expires, avoiding redundant network calls in subsequent runs.
- **Retry Logic:** To handle any connection instabilities, a *retry_session* object has been configured that automatically attempts the request up to 5 times if it fails, with an exponential backoff.

The raw data returned in binary/JSON format is then parsed and converted into a DataFrame, ready to be merged with the mobility data based on the time timestamp.

3.2 NBA Program – New York Knicks (Dynamic Web Scraping)

To acquire the match schedule, it was necessary to implement a *web scraping* procedure. Methodologically, this technique is divided into two main approaches:

- **Static Scraping:** It is based on simply downloading the page's source code (via HTTP requests). It's very fast, but it can't handle elements that require interaction (like banners) or that load later.
- **Dynamic Scraping:** Simulates the behavior of a real user by opening a browser instance (browser automation). It's slower, but it ensures you see the page exactly as it appears on video, complete with all the data and after handling any pop-ups.

In the project, the **dynamic** approach was chosen using the **Selenium** library. This choice ensures greater robustness: it allows you to wait for the data table to actually load and automatically manage the cookie banner, which would otherwise hinder reading.

Synchronization and Navigation The code controls a Chrome driver to navigate to the URLs of the seasons of interest (e.g., */teams/NYK/2025_games.html*). To avoid errors due to slow connections, an explicit wait mechanism (*WebDriverWait*) has been inserted that ensures that the main content is available before proceeding.

Table Extraction Once the page is fully visible in the automatic browser, the content is read and transformed into a DataFrame. Since we are using a dynamic approach, the table is already ready to be read by locating it using its HTML identifier (*id="games"*). The raw datset presented the following rows:

- *Date*: Game date (original format e.g., "Fri, Nov 1, 2024") (String).
- *Start (ET)*: Tip-off time in Eastern Time zone (String).
- *Unnamed: 5*: Home/Away Indicator. Empty implies "Home" (Madison Square Garden), while "@" implies "Away" (String).
- *Opponent*: Name of the opposing team (String).
- *Attend.*: Number of spectators (Attendance) present at the game (Integer).
- *Tm*: Points scored by the Knicks (Team) (Integer).
- *Opp*: Points scored by the Opponent (Integer).
- *Unnamed: 8*: Overtime flag (e.g., "OT", "2OT"), indicates if the game exceeded standard regulation time (String).

The raw dataset, derived from parsing HTML tables, has several columns labeled as *Unnamed*. While some are worthless formatting artifacts (e.g., *Unnamed: 3*, *LOG*), the *Unnamed: 5* column is critical to business logic, as it discriminates between matches played at *Madison Square Garden* (Home) and away matches. Only the former will have a spatial impact on the New York Bike Sharing system and will be maintained in the cleaning phase.

4 Data Profiling e Cleaning

The acquired raw data was subjected to a preprocessing phase to transform it into analyzable formats. The cleanup operations were differentiated based on the nature of the source (transactional, API, web scraping).

4.1 CitiBike Trip Data Cleaning

Following the data acquisition phase, the raw dataset consisted of 53 distinct CSV files. To create a unified and analytical dataset, a multi-stage preprocessing pipeline was implemented, prioritizing computational efficiency and data quality.

4.1.1 Attribute Selection

To optimize memory usage during the import process, the analysis focused exclusively on temporal metrics and user classification. Consequently, the *usecols* parameter within the read function was utilized to ingest only the relevant features:

- *started_ at*
- *ended_ at*
- *member_ casual*

4.1.2 Iterative Processing

Given the volume of the data, a monolithic import was unfeasible. An iterative approach was adopted using a control loop structure. This method allowed for the preprocessing of each CSV file individually, performing data cleaning and transformation at the single-file level before the final consolidation.

4.1.3 Data Transformation and Feature Engineering

Within each iteration, several transformation steps were executed to derive meaningful analytical variables:

1. **Type Conversion:** The *started_ at* and *ended_ at* features were converted to datetime objects to facilitate temporal arithmetic.
2. **Feature Engineering:** A new variable, *duration_ min*, was computed by calculating the difference between the end and start timestamps, expressed in minutes.
3. **Data Filtering:** To ensure data integrity, a filtering rule was applied to exclude potential recording errors or outliers. Only trips with a duration between 1 and 180 minutes were retained; records exceeding the three-hour threshold were considered anomalous and removed.
4. **Binarization:** The categorical variable *member_ casual* was encoded into two binary features: *member_ count* and *casual_ count* (assigned a value of 1 based on the user type and 0 otherwise), facilitating subsequent quantitative aggregation.
5. **Temporal Extraction:** Date and hour components were extracted from the timestamps. Specifically, the time was floored to the nearest hour and formatted as '%H:%M' to serve as a grouping key.

4.1.4 Aggregation and Consolidation

Rather than merging raw transactional records, the data was aggregated to an hourly resolution. For each file, records were grouped by date and hour to compute intermediate sums of total trips, total duration, and user counts. Finally, these intermediate results were concatenated into a single DataFrame and re-aggregated to ensure consistency. This process yielded the final metric, *avg_duration_min*, derived from the consolidated sums of duration and trip counts.

4.2 Weather Data Cleaning

Although data from structured APIs such as *Open-Meteo* has higher intrinsic quality than sources obtained through web scraping, the data profiling phase highlighted the need for specific normalization interventions to ensure semantic and syntactic consistency with the primary dataset related to the *CitiBike* service. The process of cleaning meteorological data was divided into two main phases.

4.2.1 Schema Normalization and Type Enforcement

The JSON payload, when converted to a DataFrame, had some inaccuracies in the numeric formats and column metadata.

- **Schema Normalization**: Column name normalization was evaluated but not applied, as the schema returned by the Open-Meteo API is already stable and syntactically consistent.
- **Conversion and Rounding**: Continuous meteorological values (temperature and wind speed) showed *false precision* due to floating-point representation. For continuous meteorological variables, a uniform rounding to one decimal place (*round(1)*) was applied. Numerically encoded categorical variables, such as *weather_code*, were instead converted to integer type, reducing uninformative numerical noise.

4.2.2 Timezone Alignment

The main criticality that emerged concerned the temporal alignment between the different sources of information. Weather timestamps were initially generated in Coordinated Universal Time (UTC) during the ingestion phase (using the *utc=True* parameter) and subsequently converted to datetime *naive* objects, while CitiBike transactional logs are recorded in local time (*America/New_York*).

- **Timezone Removal**: In order to enable the join operation between the datasets, the weather timestamps were converted and deprived of the timezone information via the *dt.tz_localize(None)* operation, obtaining a datetime *naive* format compatible with that of the mobility dataset.

4.3 NY Knicks Data Cleaning

The raw dataset obtained through dynamic scraping presented several structural impurities typical of HTML tables - e.g. repeated headers, non-standard data formats - and required specific feature engineering to be integrable with mobility data. The cleaning process was divided into three phases.

4.3.1 Structural Cleaning and Filtering

The first operation involved removing the syntactic noise introduced by the *Basketball Reference* structure. The raw table contained repeating headers within rows (used by the website for readability) that disrupted data continuity. A logic filter was implemented to identify and exclude rows where the column value "Rk" (Rank) or "Opponent" matched the column header itself. At the same time, an attribute selection was performed, discarding columns not relevant to the analysis - such as detailed game statistics or text notes -, retaining only the essential features: date, time, opponent, and the home/away flag.

4.3.2 Feature Engineering: Home/Away Logic

A specific criticality of this data source was the implicit representation of the event location. In the raw dataset, the "Home vs. Away" information was encoded in the *Unnamed: 5* column by the presence or absence of the "@" character. Conditional logic has been implemented to make this information explicit in a new categorical variable 'home_away': If the cell contains "@", the game is classified as "away". If the cell is empty, the game is classified as "home", so, played at Madison Square Garden. This distinction is fundamental to the project's business logic, as only "home" matches spatially influence the New York Bike Sharing system.

4.3.3 Time Parsing and Rounding (Granularity Standardization)

To allow join with CitiBike hourly data, it was necessary to standardize the temporal granularity.

1. **Analysis:** The *Date* and *Start (ET)* columns (e.g., "7:30p") were cleaned using regular expressions to remove non-standard suffixes and converted to full datetime objects.

2. **Removing non-parseable records:** Rows with uninterpretable dates or times have been deleted by forcing the conversion of invalid values to NaT and filtering them from the dataset.

3. **Rounding (Ceiling/Floor):** Since matches often start in non-standard minutes (e.g. 7:10 PM or 7:40 PM), while mobility data is aggregated into fixed time slots (e.g. 7:00 PM, 8:00 PM), a threshold-based rounding rule has been applied. Start times with minutes less than or equal to 30 were reset to the current time, while those with minutes greater than 30 were rounded to the next hour (e.g. 19:40 $\rightarrow$ 20:00).

The final result is a clean dataset ('df_knicks') containing the temporal synthetic primary key, ready for integration.

5 Data Integration

To construct a unified analytical dataset, the three heterogeneous data sources were standardized to a **common hourly temporal granularity**. This step was a fundamental prerequisite for the subsequent consolidation phase.

Consequently, a data integration strategy was implemented to synthesize a single dataset that incorporates attributes from all three distinct sources.

5.1 Granularity Standardization

The most important component of the Data Integration strategy is **Granularity Standardization**. This process involves aligning disparate datasets to a uniform level of detail; as a result, they can be meaningfully compared, consolidated, and analyzed. This standardization was achieved through specific transformations applied during the data acquisition and cleaning phases:

- **City Bike Data**: The raw transactional dataset was aggregated by hour, using a *groupby()* operation. Metrics were calculated for each hourly bucket.
- **Weather Data**: Data were retrieved via API with an explicit hourly resolution. Timestamps have been converted to datetime naive format to ensure compatibility with the local time of CitiBike data.
- **Knicks Games Data**: Game start times were rounded to the nearest hour using a threshold (flooring minutes ≤ 30, ceiling minutes > 30) to map each match to a specific slot in the timeline.

5.2 Primary Key Generation (*pk_ dt*)

To perform a deterministic join, a Surrogate Primary Key (pk_dt) was generated for every record across all datasets

$$\textit{Timestamp: } 2024-11-01\ 19:00 \rightarrow pk_dt : 2024110119 \tag{1}$$

5.3 Investigation of Correspondences

Before merging, we verified the temporal alignment between the primary dataset (Citi Bike) and the auxiliary sources. The goal was to ensure that for every hourly bucket in the traffic data, valid corresponding features existed.

5.3.1 Consistency: Bike vs. Weather Data

We checked for any missing meteorological records corresponding to bike trip hours. A set difference operation was performed to identify gaps.

Result: The output `"total coverage"` confirmed a 100% overlap (No missing weather data for the analysis period).

5.3.2 Consistency: Bike vs. Knicks Events

For the event dataset, we verified that all scraped games fell within valid hours present in the bike dataset using set operations.

Result: The check returned an empty set (or total coverage), confirming that every scraped game corresponds to a valid hourly slot in the bike dataset, ensuring a safe left join.

5.4 Data Consolidation Strategy

After standardizing the temporal granularity, the integration process focused on constructing a comprehensive analytical dataset. This phase was divided into two logical steps: the semantic enrichment of the event data and the physical consolidation of the three sources via relational joins.

5.4.1 Data Enrichment: Temporal Window Expansion

A simple point-in-time association—matching solely the game start hour—was insufficient to capture the impact of a sporting event on city mobility. Traffic anomalies typically occur during the hour preceding the match or in the period immediately following it.

To address this, we implemented a **Temporal Window Expansion** algorithm on the Knicks dataset.
For every scheduled game, the pipeline generates a set of "active" hours covering a broader window:

- **T - 1**: One hour prior to the game start (Arrival phase).
- **T**: The game start hour.
- **T + 1** and **T + 2**: The duration of the match and the subsequent exit phase.

This transformation expands the event dataset from a list of single timestamps to a continuous timeline of "event-influenced" hours.

5.4.2 Execution of Merges

The final consolidation was performed using a **Left Join** strategy, designating the *Citi Bike* dataset as the master table (left operand). This approach ensures that the continuous timeline of mobility demand is preserved, while environmental and social contexts are attached only where available.

The merging process followed a sequential order:

1. **Bike + Weather:** Integration of temperature, precipitation, and wind data based on the `pk_dt` key.
2. **Bike/Weather + Events:** Integration of the expanded Knicks dataset.

Post-merge, specific handling for missing values was applied: hours with no matching game record were explicitly filled with `0` (for the binary flag `is_knicks_event`) and labeled as `"No Game"` for the opponent feature, identifying them as standard baseline traffic periods.

5.5 Target Variable Construction

The primary objective of this phase was to construct a dataset suitable for supervised machine learning tasks. Consequently, a specific target variable was derived to represent the varying levels of service demand.

The continuous feature *total_trips* was discretized into three distinct classes based on descriptive statistical metrics, specifically utilizing the first (Q_1) and third (Q_3) quartiles of the distribution:

- **Class 0 (Low):** Demand volume below the 25th percentile ($< Q_1$).
- **Class 1 (Medium):** Demand volume within the interquartile range ($Q_1 \leq x \leq Q_3$).
- **Class 2 (High):** Demand volume above the 75th percentile ($> Q_3$).

6 Data Quality Assessment

Data quality assessment was not treated as an a posteriori phase, but as a continuous process integrated into the data management pipeline. Based on the standard data quality dimensions, specific metrics and corrective actions were implemented to ensure the reliability of the final Analytical Base Table.

6.1 Completeness and Strategy "Fixing at Source"

The completeness dimension measures the coverage of the necessary information with respect to the universe of analysis. During Data Integration, a blocking criticality emerged: a 5-hour "blind spot" in the meteorological time series at the beginning of the observation period on November 1°, caused by the misalignment between the UTC time zone of the APIs and the local time EST of the transactional logs. To manage this problem, instead of resorting to synthetic imputation techniques (e.g. averaging or interpolation) that would have introduced noise ("Usefulness vs Faithfulness"), the acquisition pipeline was modified (section 1.2 of the code). The download window was extended to the previous day (October 31), ensuring that, after the time zone conversion, all mobility time buckets had a true weather match.

6.2 Syntactic and Semantic Accuracy

Accuracy was ensured through logical filters aimed at removing artifacts and implausible values ("Outliers Detection").

- **Physiological Filter (Bike Data)**: by analyzing the distribution of trip duration, technically valid but semantically incorrect records were identified (e.g. trips < 1 minute or > 180 minutes). These typically represent station docking errors or thefts/losses, not a real mobility demand. Excluding these records improved the semantic accuracy of the 'total_trips' target.
- **Structural Cleaning (Knicks Data)**: the syntactic accuracy of the web-scraped dataset has been improved by removing repeated header rows within the HTML table and handling implicit values (e.g. converting the "@" flag to an explicit categorical variable "Home/Away").

6.3 Temporal Consistency and Granularity

Consistency refers to the consistency of data representation across different sources.

- **Timezone Alignment:** A time consistency was forced by converting all timestamps to *naive datetime* format (removing the UTC timezone from weather data) to align with the New York local time used by CitiBike.
- **Granularity Standardization:** To resolve the granularity conflict between point events (game start e.g. 7:10 PM) and time buckets (e.g. 7:00-8:00 PM), a rounding function (Ceiling/Floor logic) was applied. This ensured that each event was uniquely associated with the correct mobility time interval, preserving referential integrity during the Join.

6.4 Referential Uniqueness and Integrity

To mitigate the risk of duplication typical of iterative pipelines, a unique synthetic Primary Key ('pk_dt': YYYYMMDDHH) was generated for each hour of observation. Before the storage phase, a deduping logic ('drop_duplicates') based on this key is applied, ensuring that no redundant records exist in the final Database for the same time slot.

7 Data Storage

7.1 Data Model Strategy

The storage architecture adopted for the project is based on the **Relational Model**, a logical data model in which information is organized into tables characterized by an explicit pattern of rows and columns. This model provides a structured, consistent, and deterministic representation of data, making it particularly suitable for the **data management** phase of supervised machine learning pipelines.

In the project context, using a relational model allows each temporal observation to be represented as a single row and each attribute as a column, ensuring consistency between features, temporal alignment, and uniqueness of observations through the synthetic primary key *pk_dt*. This structure is functional to the construction of an *Analytical Base Table*, a fundamental prerequisite for subsequent analysis and modeling activities.

The logic model is implemented via a **Relational Database Management System (RDBMS)**, which offers strong guarantees of structural data integrity. In particular, the consistency and atomicity properties of the relational model help preserve the reliability of the dataset during incremental update and deduplication operations, avoiding the presence of partial or inconsistent records.

Although RDBMS solutions have limitations compared to NoSQL architectures, these constraints do not penalize the project scope. The volume of analyzed data does not fall within the *Big Data* scenario and does not require horizontal scalability mechanisms. On the contrary, the rigidity of the scheme represents an advantage, since the input data (timestamps, meteorological variables, event indicators) are naturally structured and perfectly align with the tabular paradigm.

7.2 Archiving Implementation

The physical implementation of the data model was achieved using the **SQLite** engine, chosen as the lightweight relational storage solution. SQLite is used as an intermediate analytical storage layer, supporting data integration, validation, and preparation tasks, and not as a data warehousing or serving system.

Persistence of the final integrated dataset was achieved using a dedicated Python function, `save_incremental_to_sqlite`, which leverages the pandas library's `to_sql` method to store data in the local database **nyc_mobility.db**. The routine implements an incremental update strategy, merging new records with existing ones and ensuring the uniqueness of observations via deduplication logic based on the primary key *pk_dt*.

This choice allows for reproducible, queryable, and easily validable storage via SQL queries, supporting consistency, aggregation, and dataset auditing checks before any downstream use. Overall, adopting SQLite-based relational storage represents an effective compromise between structural guarantees, operational simplicity, and consistency with project data management objectives.

8 Analytical Validation

The final step of the pipeline involved a rigorous validation of the stored data. Verifying the database via direct SQL querying is essential not only to confirm the technical success of the storage operation but also to assess the *semantic consistency* of the integrated dataset.

8.1 Query home games

In order to validate the event integration, we aggregated the data by the *is_knicks_event* flag to verify the presence of the events and their impact on traffic volume.

```
SELECT
    CASE
        WHEN is_knicks_event = 0 THEN 'No game'
        WHEN is_knicks_event = 1 AND home_away = 'home' THEN 'home
           game'
        WHEN is_knicks_event = 1 AND home_away = 'away' THEN 'away
           game'
        ELSE 'Altro'
    END as event_type,
    COUNT(*) as total_hours,
    ROUND(AVG(total_trips), 0) as avarage_hourly_trips
FROM abt_classification
GROUP BY event_type
ORDER BY avarage_hourly_trips DESC;
```

Listing 1: Validation: Event Impact Analysis

Console Output:

```
query1:
 event_type  total_hours  avarage_hourly_trips
```

```
 home game            85                6969.0
 away game            86                6107.0
   No game          8588                5253.0
```

The output confirms the successful integration between the mobility and event datasets (Knicks games).
The aggregation metrics provide empirical evidence of a distinct stratification in traffic intensity: *Home Games* correspond to the peak average demand ($\approx$ 6,969 trips/hour), representing a significant increase (+32%) compared to the baseline activity of *No Game* intervals ($\approx$ 5,253 trips/hour).
Moreover, the ability to isolate and quantify this deviation demonstrates that the merge operation correctly aligned the temporal dimensions of both sources, effectively capturing the impact of sporting events on urban mobility.

8.2 Query Weather

The second validation focused on the physical relationship between environmental conditions and mobility demand. We aggregated the average rainfall and temperature for each target class (Low, Medium, High Demand), introducing specific ranges to distinguish between light, moderate, and heavy rain.

```
SELECT
CASE
    WHEN rain_mm = 0 THEN 'Clear'
    WHEN rain_mm > 0 AND rain_mm < 2 THEN 'Light Rain'
    WHEN rain_mm >= 2 AND rain_mm < 7 THEN 'Moderate Rain'
    WHEN rain_mm >= 7 THEN 'Heavy Rain'
END as weather_condition,
    ROUND(AVG(total_trips), 0) as avarage_trips
FROM abt_classification
GROUP BY weather_condition
ORDER BY avarage_trips DESC;
```

Listing 2: Validation: Weather vs Avarage Trips

Console Output:

```
query2:
 weather_condition  avarage_trips
             Clear         5445.0
       Light Rain          4350.0
       Heavy Rain          4239.0
    Moderate Rain          3667.0
```

The output substantiates the negative correlation between rainfall intensity and trip volume. The breakdown reveals that *Moderate Rain* conditions correspond to a consistent reduction in trips compared to *Light Rain* across all demand segments. This trend is

particularly strictly linear in the **High (2)** and **Low (0)** demand classes (defined by the quantile-based `classify` function), where the average volume progressively decreases from *Clear* to *Heavy Rain*, confirming that precipitation intensity acts as a strong disincentive for mobility.

8.2.1 Correlation analysis as diagnostic validation

As an additional diagnostic validation step, a correlation analysis among numerical features was performed on the integrated dataset. The analysis was used to verify the presence, direction and coherence of relationships between the target variable and selected exogenous features, without introducing feature selection or model-based assumptions.

The results confirm temperature as the most relevant exogenous driver of demand, showing a moderate positive correlation with total trips (Pearson correlation coefficient $\rho \approx 0.42$). Precipitation and humidity exhibit negative correlations ($\rho \approx -0.06$ and $\rho \approx -0.25$, respectively), supporting the hypothesis that adverse weather conditions act as a disincentive to bike usage. The event indicator shows a weak but positive correlation ($\rho \approx 0.07$), consistent with the localized and time-dependent nature of event-driven demand. The strong correlation observed between total trips and the demand class ($\rho \approx 0.88$) is expected, as the target variable is directly derived from the underlying volume distribution.

8.3 Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) constitutes a critical diagnostic phase in the data management pipeline. By statistically and visually inspecting the integrated dataset, we validate the integrity of the data fusion process and examine the underlying distribution of the target variable and its relationship with exogenous features (i.e., weather and events), supporting data quality assessment and integration validation (see Figures 1 and 2).

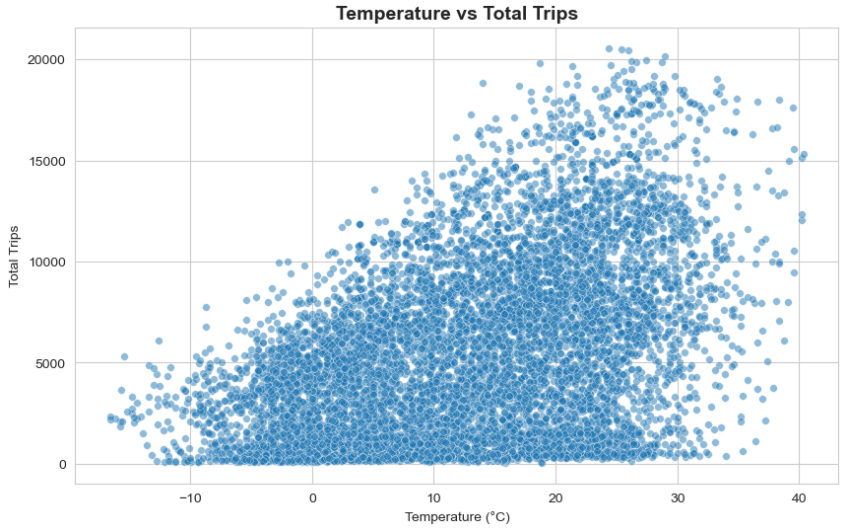


Figure 1: Relationship between air temperature and total hourly CitiBike trips.

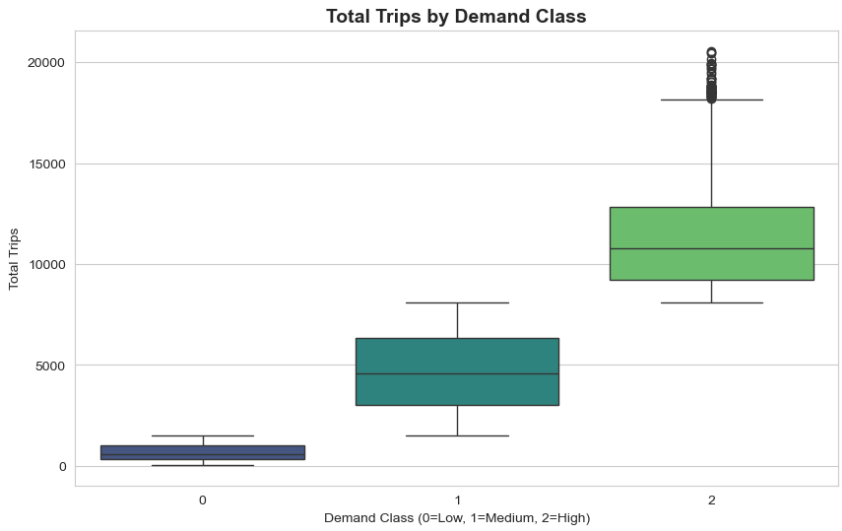


Figure 2: Distribution of total hourly trips across demand classes (Low, Medium, High).

9 Conclusions

The project implemented a comprehensive data management architecture, which governs the entire **Data Lifecycle** of urban mobility information: from the *Data Acquisition* phase to the final *Data Storage* phase.

9.1 Lifecycle Management and Integration

The pipeline successfully addressed the challenges typical of multi-source systems described in the theory:

- **Data Acquisition and Heterogeneity:** The *Heterogeneity* of sources was handled by acquiring structured data (transactional CSVs), API data (Weather), and semi-structured data from the Web (HTML). The use of robust techniques (sessions with *retry backoff*) ensured the continuity of the input flow.
- **Data Preparation and Conflict Resolution:** In the preparation phase, *Data Conflicts* were resolved at the instance level. Specifically, time zone normalization and time granularity standardization eliminated inconsistencies between datasets, allowing for correct **Object Identification** (the deterministic correspondence between sporting events and hourly traffic flows).
- **Data Quality:** The process improved the data quality dimensions, specifically **Completeness** (retrieving missing weather data by extending the acquisition window) and **Semantic Accuracy** (filtering implausible trip duration values, such as $duration > 180$ min).

Validation via data storage and analysis The final data, transformed into **Reconciled Data**, was historicized according to the *Relational Model* on SQLite. By querying this repository, analytical results were highlighted that validate the quality of the integration process:

- **Event Correlation:** Coherently to initial assumptions, analysis of the integrated data shows that sporting events (*Home Game*) generate an average traffic volume (**6,969 trips/hour**) **32%** higher than the baseline scenario (*No Game*, 5,253 trips/hour). This confirms that the enrichment logic correctly isolated the signal.

- **Exogenous variables:** The correlation matrix confirms **Temperature** as the main driver of demand (positive correlation **0.41**), validating the hypothesis that environmental conditions directly influence user behavior.

In summary, the riser architecture demonstrates how heterogeneous raw data can be transformed into high-quality, reconciled data, ready for advanced analytics and for use in the data management phase of supervised machine learning pipelines.

In this project, Artificial Intelligence was used as a support tool for writing parts of the code and for refining the language of the report. The analysis, design choices, and interpretation of the results are entirely attributable to the author.

[0]In this project, Artificial Intelligence was used as a support tool for writing parts of the code and for linguistic refinement of the report texts.